

Раздел IV

ПРОГРАММИРОВАНИЕ АЛГОРИТМОВ НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ PYTHON (Пайтон)

4.1. Одномерный массив

Вы научитесь:

– создавать программы на языке программирования Python (пайтон) с использованием одномерных массивов.

Ключевые слова

Список

Массив

Одномерный массив

Элемент

Индекс

Тізім

Жиым

Бір өлшемді жиым

Элемент

Индекс

List

Array

One-dimensional array

Element

Index



Дайте общую трактовку ключевых слов и предположите, как они будут фигурировать в последующем тексте по теме.



Предположим, что вам необходимо обработать информацию о курсе тенге в долларах США с 20 по 31 декабря 2018 года (табл. 4.1).

Каким образом можно разместить данные на каждый день? Как можно обработать данные за несколько лет?



Приведем пример размещения данных из табл. 4.1 в отдельные переменные day1, day2, ..., day11.

Таблица 4.1

Курс тенге к доллару

Дата	Курс тенге к 1 доллару	Данные в отдельных переменных	Код программы	Результат
20.12.2018	372,49	day1 = 372.49; print (day1)	<pre>File Edit Format Run Options Win day1 = 372.49; print (day1) day2 = 370.76; print (day2) day3 = 371.35; print (day3) day4 = 371.35; print (day4) day5 = 371.35; print (day5) day6 = 371.68; print (day6) day7 = 371.08; print (day7) day8 = 375.15; print (day8) day9 = 380.44; print (day9) day10 = 380.44; print (day10) day11 = 384.20; print (day11)</pre>	<pre>>>> 372.49 370.76 371.35 371.35 371.35 371.68 371.08 375.15 380.44 380.44 384.2 >>></pre>
21.12.2018	370,76	day2 = 370.76; print (day2)		
22.12.2018	371,35	day3 = 371.35; print (day3)		
23.12.2018	371,35	day4 = 371.35; print (day4)		
24.12.2018	371,35	day5 = 371.35; print (day5)		
25.12.2018	371,68	day6 = 371.68; print (day6)		
26.12.2018	371,08	day7 = 371.08; print (day7)		
28.12.2018	375,15	day8 = 375.15; print (day8)		
29.12.2018	380,44	day9 = 380.44; print (day9)		
30.12.2018	380,44	day10 = 380.44; print (day10)		
31.12.2018	384.20	day11 = 384.20; print (day11)		

Сколько переменных потребуется для размещения данных в программе?

Для каждой переменной (day1, day2, ..., day11) в памяти резервируется область, которая похожа на контейнер. В ней хранятся ссылки на значение (рис. 4.1), а не содержат его, как во многих языках программирования. Данный способ представления данных позволяет работать в программе с **отдельными переменными**.



Рис. 4.1. Контейнеры

Как можно обрабатывать информацию в **наборе переменных**? Для этого в языке программирования Python применяются **списки**, являющиеся аналогом термина «массив» в большинстве языков программирования. **Массив** может состоять из одной строки и нескольких столбцов, которые являются элементами массива. Такой массив называется **одномерным**.

Так как массивы в языке программирования Python представляют собой список элементов, то в дальнейшем при работе с одномерными массивами будем применять термин «списки».



Список (list) – это один из типов последовательностей в языке Python. В последовательностях хранятся упорядоченные наборы объектов, объединенные под общим именем.

Переменные, входящие в состав списка, называются **элементами**. Элементы списка заключены в квадратные скобки и разделены запятыми.



Зададим список с именем *s* (рис. 4.2), применяя данные из табл. 4.1:

```
s = [372.49, 370.76, 371.35, 371.35, 371.35, 371.68, 371.08, 375.15, 380.44, 380.44, 384.20]
```

The screenshot shows a Python IDE window with the following content:

```
File Edit Format Run Options Window Help
s = [372.49, 370.76, 371.35, 371.35, 371.35, 371.68, 371.08, 375.15, 380.44, 380.44, 384.20]
print(s)
```

A callout box labeled "Код программы" (Program Code) points to the code above. Below the code, the output is shown:

```
[372.49, 370.76, 371.35, 371.35, 371.35, 371.68, 371.08, 375.15, 380.44, 380.44, 384.2]
>>> |
```

A callout box labeled "Результат" (Result) points to the output line.

Рис. 4.2. Ввод данных списком

Для эффективной работы с последовательностью элементов используется **индекс**. С его помощью описывают местоположение и обращение к элементу списка, заключенному в квадратные скобки. Нумерация индексов в списках начинается с нуля. Первому элементу последовательности присваивается индекс 0, второму – индекс 1, третьему – индекс 2, и т. д. Например: *s*[0]=372.49, *s*[1]=370.76, *s*[2]=371.35, ... *s*[10]=384.20.

Особенностью элементов списка в Python является то, что они могут быть разного типа. Например, в списке *lst*=[5, 3.14, 'hello', 's'] применяются данные целого и вещественного типов, строки (рис. 4.3).

The screenshot shows a Python IDE window with the following content:

```
File Edit Format Run Options Window
lst=[5, 3.14, 'hello', 's']
print(lst)
```

On the right side, the output is shown:

```
Type "copyright", "credits"
>>>
===== RESTART: C:/Users/TAC/
[5, 3.14, 'hello', 's']
>>>
```

Рис. 4.3

Списки относятся к изменяемым типам. Это означает, что их содержимое можно добавлять, удалять и изменять без создания нового объекта. Существуют два способа создания пустых списков:

- 1) с вызовом класса **list**: `names = list ()`;
- 2) с использованием синтаксиса литералов в квадратных скобках `[]`: `names = []`.

Как создавать заполненные списки? В языке программирования Python существует для этого специальная синтаксическая конструкция, которая называется генератором списков. Например, конструкция `[i for i in range(1,10)]`, заключенная в квадратные скобки, это генератор списка.

Внутри квадратных скобок можно выделить три части, отделенные друг от друга ключевыми словами **for** и **in**:

- 1) что делаем с элементом (добавляем в список);
- 2) что берем (элемент `i`);
- 3) откуда берем (из объекта `range`).

Функция **range ()** создает последовательность (диапазон) чисел. В качестве аргументов функция принимает начальное значение диапазона (по умолчанию 0), конечное значение (не включительно) и шаг (по умолчанию 1).

Разберем с помощью примера три части:

```
a = [5, -5, 2, -2, 7, 5, 3, 8]
b = [i**3 for i in a]
print(b)
```

Берем каждый элемент из списка `a` и возводим его в генераторе списка в куб:

- 1) что делаем – возводим элемент в куб;
- 2) что берем – элемент;

`[125, -125, 8, -8, 343, 125, 27, 512]` 3) откуда берем – из списка `a`.

Благодаря генератору списка запись программного кода становится короткой, чем при создании списка классическим способом. Приведем пример (табл. 4.2), в котором надо создать список, заполненный натуральными числами от 1 до 10.

Таблица 4.2

Способы создания списков

Классический способ	Способ – генератор списка
<pre>s = [] for i in range(1,10): s.append(i) print (s) [1, 2, 3, 4, 5, 6, 7, 8, 9]</pre>	<pre>s = [i for i in range(1,10)] print (s) [1, 2, 3, 4, 5, 6, 7, 8, 9]</pre>

Для эффективного решения разнообразных задач используются **функции и операции** для работы со списками. Рассмотрим некоторые из них (табл. 4.3).

Таблица 4.3

Специальные функции и операции для списков

Функции	Назначение	Примеры
append(x)	добавляет элемент x в конец списка	<pre>a = [1, 2, 3, 4, 5] a.append([6, 7, 8, 9, 10]) print (a)</pre> Результат [1, 2, 3, 4, 5, [6, 7, 8, 9, 10]]
a.extend(s)	добавляет все элементы списка s в конец списка a	<pre>a = [1, 2, 3, 4, 5] a.extend([6, 7, 8, 9, 10]) print (a)</pre> Результат [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
sum(s)	возвращает сумму элементов в списке	<pre>s = [1,2,3,4,5] print(sum(s))</pre> Результат 15
Операции	Назначение	Примеры
lst1+lst2	Сложение или объединение списков – приписывание одного списка к другому	<pre>a = [9, 8, 7] b = [6, 5] c = a + b print(c)</pre> В результате список: c = [9, 8, 7, 6, 5]
lst * n	n кратное повторение списка lst (умножение списка lst на число n)	<pre>b = [6, 5] d = b * 3 print(d)</pre> В результате список: d = [6, 5, 6, 5, 6, 5]



Поэкспериментируйте.

Составьте программу создания и считывания списков. Напишите комментарии к каждой строке программы.

Введите код программы в IDLE – Сохраните – Запустите модуль – Выведите.



Шаг за шагом

Прежде всего нужно создать пустой список (не содержащий элементов, длины 0). В конец списка будут добавляться элементы с помощью метода **append**:

```
massiv=[]          # пустой список
for i in range(0,6): #
    a=input("Элемент массива: ")
    massiv.append(a)
print ("Исходный массив: ", massiv)
```

- С какой целью в программе используется оператор цикла **for**?
- Какую роль играют четыре пробела в программе перед операторами?
- Попробуйте убрать пробелы в программе. Каким будет результат программы?

  	Знание и понимание 1. Что такое <i>список</i>? 2. Может ли список содержать разнородные типы данных? 3. Для чего применяется имя списка? 4. Что такое <i>индекс</i>? Как начинается индексация элементов в списке? 5. Что такое <i>элемент списка</i>? Каким образом размещаются элементы в памяти? 6. Какова отличительная особенность списка в Python? 7. Какой список называется пустым? 8. Что является аналогом списка? 9. Как с помощью генератора списка можно создавать заполненные списки? 10. Для чего применяются методы в списках? Перечислите методы. 11. Составьте и запишите свою версию рассказа, употребив все предложенные ключевые термины по теме.
 	Применение Анализ 12. Какой список будет выведен на экран после выполнения кода программы: <pre>hi = ['bonjour', 'привет', 'hola', 'aloha', 'salem'] hi[1] = 'hello' print(hi)</pre> 13. Что будет выведено на экран после выполнения блока функций? <pre>print([4, 3] + [2]) print([0, 1] * 2)</pre>

14. Дана программа в Python:

```
a = [] # пустой список
n = int(input()) # количество элементов в списке
for i in range(n):
    new_element = int(input()) # считывается очередной элемент
    a.append(new_element) # добавляется элемент в список
print(a)
```

– Введите код программы в **IDLE** – **Сохраните** – **Запустите модуль** – **Выведите**.

- Проанализируйте, как будет происходить считывание списка.
- Какое количество элементов в списке будет задано на входе?
- Сколько элементов списка будет находиться в отдельной строке?
- Куда будут добавляться элементы с помощью метода `append`?

15. Дан код программы:

```
numbers = [1, 2, 3, 4, 5]
strings = ['казахские', 'традиции:', 'асар', 'жылу', 'конакасы', 'конаккаде']
mylist = [numbers, strings]
print(mylist)
```

- 1) Сколько переменных созданы в программе?
- 2) Какие типы данных хранятся в списках?
- 3) Какой результат будет выведен на экране?
- 4) Сколько элементов будут храниться в третьем списке?



Синтез

Оценивание

16. Каким образом можно заполнить список числами, кратными 3 или 5, чтобы на экран был выведен список: [3, 5, 6, 9, 10, 12, 15, 18, 20]?

17. Ниже приведена программа, в которой генерируется список случайных целых чисел. Напишите комментарии к программе. Определите, что будет выведено на экран в результате выполнения данной программы.

```
import random
s = []
for i in range(10):
    s.append(int(random.random() * 100))
print(s)
chet = 0
nechet = 0
for i in s:
```

```

if i % 2 == 0:
    chet += 1
else:
    nechet += 1
print ("Chet:", chet)
print ("Nechet:", nechet)

```

18. Оцените преимущества и недостатки использования списков при обработке большого набора данных.



Создание списка разными способами.

Приведем примеры создания списка разными способами.

1. Создание списка, заполненного одинаковыми элементами с использованием оператора повторения списка:

A = [0] * n

2. Создание списков, заполненных по сложным формулам с использованием генераторов, – выражения, позволяющие заполнить список некоторой формулой. Общий вид генератора следующий:

[выражение for переменная in список]

где переменная – идентификатор некоторой переменной, список – список значений.

2.1. Создать список, состоящий из n нулей с помощью генератора:

A = [0 for i in range (n)]

2.2. Создать список, заполненный квадратами целых чисел:

A = [i ** 2 for i in range(n)]

2.3. Заполнить список квадратами чисел от 1 до n с изменением параметров функции range на range (1, n + 1):

A = [i ** 2 for i in range (1, n + 1)]

2.4. Список, заполненный случайными числами от 1 до 9 (используя функцию randint из модуля random):

A = [randint(1, 9) for i in range(n)]

2.5. Список, считанный стандартным вводом, – ввод числа элементов списка, которое будет использовано в качестве аргумента функции range, затем ввод заданного количества строк:

A = [input() for i in range(int(input()))].

4.2. Поиск элемента с заданными свойствами

Вы научитесь:

– создавать программы на языке программирования Python (пайтон) с использованием одномерных массивов.

Ключевые слова

Поиск элемента	Элементті іздеу	Finding an item
Алгоритм поиска	Іздеу алгоритмі	Search Algorithm
Линейный поиск	Сызықтық іздеу	Linear search



Дайте общую трактовку ключевых слов и предположите, как они будут фигурировать в последующем тексте по теме.



Очень часто при обработке информации необходимо решать задачи по поиску данных с определенными свойствами. Поиск информации является одним из наиболее часто выполняемых алгоритмов. Задачи по поиску данных разнообразны. Например, поиск данных разыскиваемого человека, телефонного номера, необходимого товара, поиск школы, угадывание числа в играх и т. д.



Поиск – это обработка некоторого множества данных с целью выявления подмножества данных, соответствующего критериям поиска.



Подумайте и приведите примеры использования алгоритмов поиска информации из вашей практики.



К основным алгоритмам, используемых для поиска элементов в списке, относятся (рис. 4.4):

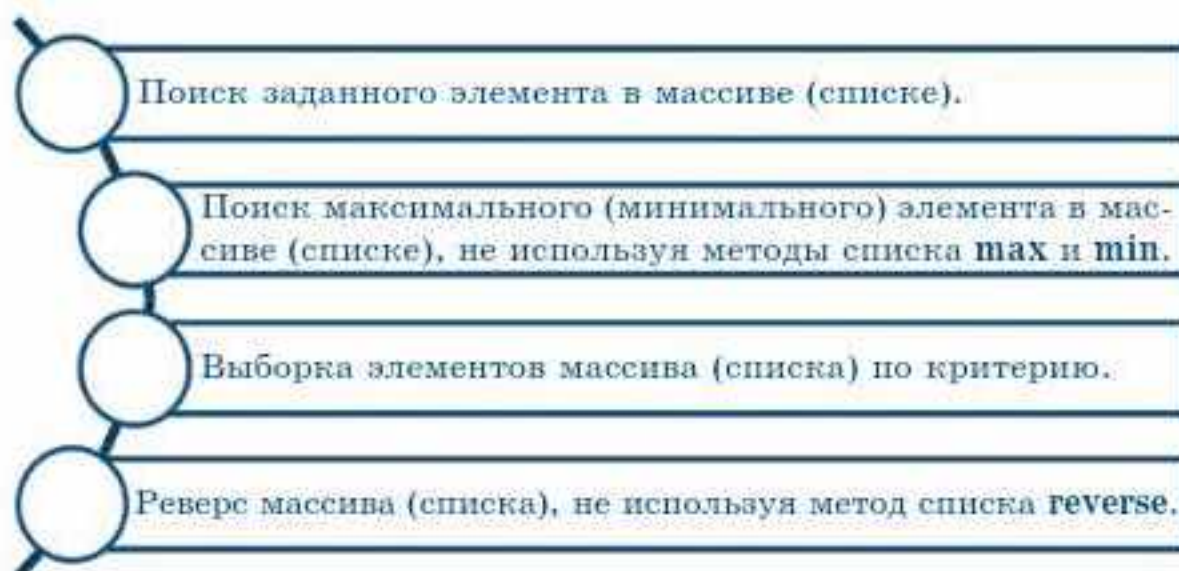


Рис. 4.4. Методы поиска элементов в списках

Самый простой способ поиска элементов списка – **линейный**, или **последовательный**. При линейном поиске последовательно проверяют все элементы списка и сравнивают их с ключевым значением.



Рассмотрим, как осуществляется поиск элементов в программах на Python.



1. Код программы поиска заданного элемента в списке.

Код программы	Алгоритм выполнения
<pre># задаем список arr = [100, -355, 111, -765, 123, -99] # исходное значение point = 111 # длина списка или количество элементов l = len(arr) for i in range(l): if arr[i] == point: print ("заданный элемент – содержит") break else: print ("заданный элемент – не содержит")</pre> Результат заданный элемент – содержит	Для проверки заданного элемента, входящего в список, применим цикл for. Каждый элемент списка сравниваем со значением в переменной point (эталон). В условном операторе if проверяем условие. Если заданное число совпадает с элементом в списке, то выводится соответствующее сообщение: «заданный элемент – содержит», и мы выходим из цикла. Если элементы списка последовательно проверены и совпадения отсутствуют, то выполняются else и команда print ("заданный элемент – не содержит").

2. Поиск максимального (минимального) элемента в списке.

В жизни для решения задач часто необходимо найти **максимальный** (наибольший) или **минимальный** (наименьший) элемент массива. Например, определить рост самого высокого ученика в классе, какого числа была самая низкая температура зимой в Алматы, самую большую площадь территории среди евразийских стран. Ниже приведен пример, в котором выполняется поиск максимального элемента списка.

Код программы	Алгоритм выполнения
<pre># задаем список arr = [22, 8, 27, -55, 0, 15, 42, -5, 77, 99] # максимальный элемент равен arr[0] maximum = arr[0] for i in range(1, len(arr)): if arr[i] > maximum: maximum = arr[i] print(maximum)</pre>	Максимальный элемент сравнивается последовательно со всеми элементами списка со знаком > (больше). Если найдутся элементы списка больше arr [0], то значение максимума обновится. Иначе значение максимума останется равным arr [0]. Чтобы найти минимальный элемент списка, необходимо знак поменять на < (меньше).
Результат 99	

3. Выборка элементов списка по критерию.

На языке программирования Python сложное условие **if** можно записать разными способами. Например, **if arr[i - 1] < arr[i] > arr[i + 1]**. Ниже на рис. 4.5 представлен результат программы.

```
File Edit Format Run Options Window Help
import random
#Список случайных чисел
arr = [random.randint(0, 50) for i in range(10)]
#Вывод списка на экран
print(arr)
for i in range(1, len(arr) - 1):
    if arr[i-1] < arr[i] and arr[i] > arr[i+1] :
# Вывод индексов, подходящих элементов
    print(i, end=" ")

Python 3.4.4 Shell
File Edit Shell Debug Options Window Help
Python 3.4.4 (v3.4.4:737efcadf5a6, Dec 20 2015, 20:20:57) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
----- RESTART: C:/Users/Алира/Desktop/3.py -----
[9, 19, 45, 30, 3, 21, 30, 13, 0, 17]
2 6
>>>
```

Рис. 4.5

Также можно записать сложное условие **if**, применив логический оператор **and** (рис. 4.6):

if arr[i-1] < arr[i] and arr[i] > arr[i+1].

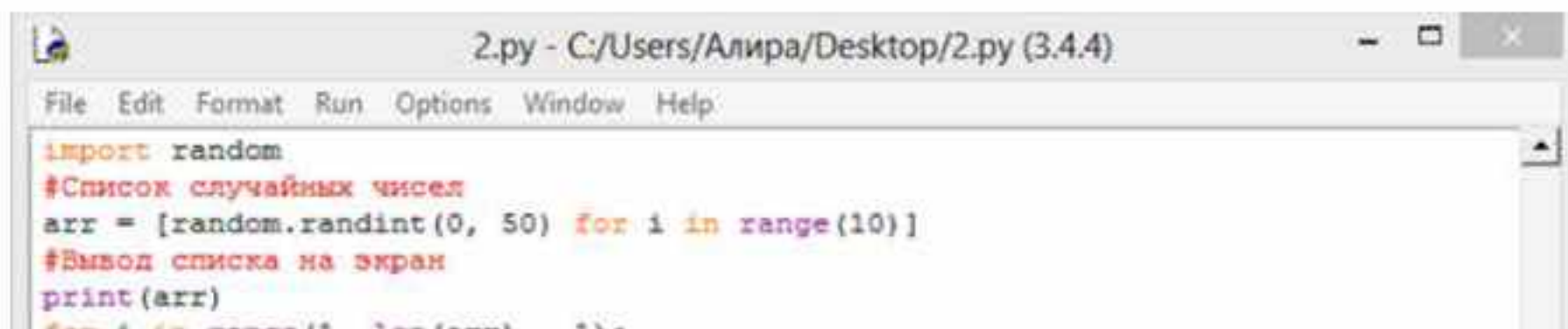


Рис. 4.6

Мы видим, что разные способы записи сложного условия `if` в Python выполняют одну и ту же функцию задания.

Код программы	Алгоритм выполнения
<pre>import random #Список случайных чисел arr = [random.randint(0, 50) for i in range(10)] #Вывод списка на экран print (arr) for i in range(1, len(arr) - 1): if arr[i-1] < arr[i] and arr [i] > arr[i+1]: # Вывод индексов, подходящих эле- ментов print (i, end=" ") Результат [16, 23, 34, 23, 29, 13, 4, 19, 49, 25] 2 4 8</pre>	Вывести номера элементов списка, которые больше стоящих по соседству. Начиная со второго элемента массива <code>arr[1]</code> и до предпоследнего, сравниваем элементы списка с соседями. Если элемент списка подходит, то на экран в одну строчку выводятся его индексы.

4. Кортежи.

Следующий тип данных (класс) – **кортеж (tuple)**. Кортеж условно называют неизменяемым «списком». К нему применимы многие списковые функции, кроме изменения. Кортежи используются, если элементы структуры данных не будут изменены в процессе работы программы.

С помощью кортежей можно присваивать значения одновременно двум переменным:

```
(x, y) = (10, 5) # результат не изменится, если убрать круглые скобки
print(x)
print(y)
```

Результат:

10

5

Кортеж нельзя изменить, но можно изменить список, входящий в кортеж:

```
kort = (1, [4, 5], '3')
print(kort[1])
```

Результат:

[4, 5]

```
kort[1][0] = '7'
print(kort)
```

Результат:

(1, ['7', 5], '3')

5. Реверс массива (списка), не используя метод списка reverse.

Код программы	Алгоритм выполнения
<pre>import random #Список случайных чисел arr = [random.randint (0, 50) for i in range (10)] #Вывод списка на экран print(arr) l = len(arr) for i in range(l//2): arr[i], arr[l - 1 - i] = arr[l - 1 - i], arr[i] print(arr)</pre>	Принцип алгоритма заключается в следующем: мы меняем местами 0-й элемент с последним, 1-й – с предпоследним и т. д. В итоге количество таких обменов будет равно половине длины списка. Элементы поменяются местами по второму кругу и вернутся в первоначальное положение. Цикл выполняется 1/2 раз (10/2).

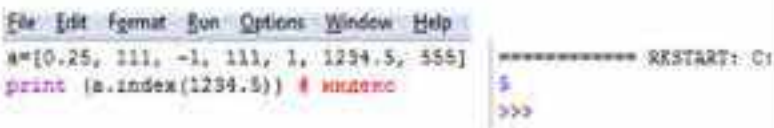
Продолжение

Результат [50, 2, 42, 40, 2, 39, 26, 0, 44, 9] [9, 44, 0, 26, 39, 2, 40, 42, 2, 50]	
--	--

Кроме операций сложения и повторения списков, с которыми вы познакомились в § 4.1, используются **встроенные функции** для списков (табл. 4.4). Например, такие функции, как **min()** и **max()**, возвращают соответственно минимальный и максимальный элементы последовательностей. Метод **index()** возвращает индекс (номер) элемента в последовательности.

Таблица 4.4

Встроенные функции для списков

Функции	Назначение	Примеры
len(list)	вычисляет общую длину списка list	print(len([1, 2, 3, 4, 5, 6, 7, 8, 9])) Результат: 9
max(list)	возвращает элемент из списка list с максимальным значением	print(max([1,2,3,4,5])) Результат: 5
min(list)	возвращает элемент из списка list с минимальным значением	print(min([1,2,3,4,5])) Результат: 1
index()	возвращает индекс элемента x. Если элемент не найден, генерирует исключение ValueError	a=[0.25, 111, -1, 111, 1, 1234.5, 555] print (a.index(1234.5)) # индекс Код программы Результат 
reverse()	расставляет все элементы в списке в обратном порядке	data = list[1,2,3,4,5,6,7,8,9] data.reverse() print(data) Код программы Результат 

Продолжим знакомство с операциями проверки вхождения элементов в список (табл. 4.5), которые часто применяются при решении задач на языке программирования Python.

Таблица 4.5

Операции работы со списком

Операция	Назначение	Примеры
<code>x in A</code>	Проверяет, содержится ли элемент в списке. Возвращает True или False	<code>print(5 in [1, 2, 3, 4, 5])</code> Результат: True
<code>x not in A</code>	Аналогично проверяет <code>not (x in A)</code>	<code>print(5 not in [1, 2, 3, 4, 5])</code> Результат: False

Задачи по программированию с использованием списков решаются, как правило, по следующему алгоритму (рис. 4.7).



Рис. 4.7. Алгоритм работы со списками

Если в процессе обработки значения элементов списка изменяются, то вывод осуществляется дважды: до и после обработки.





Знание и понимание

1. Что такое *поиск информации*?
2. Каким образом упорядочены данные в массивах?
3. Как называется самый простой способ поиска элементов в массиве?
4. Какие числа называют максимальными и минимальными?
5. Чем отличается условие поиска максимального и минимального элементов?
6. Какие встроенные функции используются для списков?
7. Почему встроенные функции облегчают написание программы?
8. Почему алгоритм работы со списками имеет одинаковую структуру?
9. Составьте и запишите свою версию рассказа, употребив все предложенные ключевые термины по теме.



Применение

Анализ

10. Дан код программы. Каким образом будет заполнен новый список в зависимости от значений исходного?

```
# исходный список
a = [17, -11, 18, 10, 0, 77, -1, 12, -5, 25]
# пустой список
b = []
# перебор элементов исходного списка
for i in a:
    if i > 0: # если элемент больше 0,
        b.append(1) # в список добавляется 1.
    elif i < 0: # если элемент меньше 0,
        b.append(-1) # в список добавляется -1.
    else: # если элемент равен 0,
        b.append(0) # в список добавляется 0.
# вывод исходного и производного списков
print(a)
print(b)
```

11. Дана строка `s = 'abc12d3e5f7g8'`, состоящая из цифр и букв. Необходимо из строки выбрать все цифры и сложить их в массив как числа `digits == [1, 2, 3, 5, 7, 8]`. Составьте программу с использованием цикла `for`.

12. Найдите произведение элементов списка, кратных 3 и оканчивающихся на 5. Если таких элементов нет, сообщите об этом.

```
import random
#Создадим список случайных чисел
arr = [random.randint(0, 50) for i in range(10)]
#Выведем его на экран
print(arr)
#При поиске произведения – присваиваем переменной значение 1
p = 1
for i in range(len(arr)):
    if arr[i] % 3 == 0 and arr[i] % 10 == 5:
        p *= arr[i]
#Если произведение равно единице, то таких элементов нет
if p == 1:
    print("Нет таких элементов")
else:
    print(p)
[46, 47, 10, 10, 20, 47, 15, 48, 6, 16]
720
[48, 22, 31, 28, 37, 35, 36, 29, 42, 44]
Нет таких элементов
```



Синтез

Оценивание

13. В ботаническом саду растут деревья разной высоты. Для исследования необходимо методом случайной выборки определить 10 деревьев, имеющих высоту от 1 до 50 метров. Затем, используя этот список, нужно осуществить поиск дерева с максимальной высотой и соответственно его индекс, а также дерева с минимальной высотой и его индекс. Составьте программу на языке программирования Python.

Алгоритм поиска максимального элемента в списке будет целого типа. Максимальный элемент можно хранить в переменной `arr[max]`, а его индекс – в переменной `max`. Простейший алгоритм поиска сводится к последовательному сравнению со значением `max` всех элементов массива `arr`, начиная со второго в цикле `for` с параметром `i`.

На каждом шаге цикла будем проверять условие `arr[i] > arr[max]`. Если оно выполнится (очередной элемент больше `max`), то значение этого элемента примем в качестве максимального `max`, а его индекс присвоим переменной `max=i`. После перебора всех элементов массива переменная `arr[max]` будет содержать значение максимального элемента массива, а переменная `i` – его индекс. Аналогично для минимального значения самостоятельно проверьте условие:

`if arr[i] < arr[min].`

14. Найдите произведение самого большого и самого маленького элементов списка.

15. Заполните список случайными числами и выполните реверс.

16. Найдите максимальный и минимальный рост 100 спортсменов в списке, в котором даны случайные числа.



Задачи для углубленного изучения

1) Задана строка: «Казахская степь занимает большую часть Северного и Центрального Казахстана, широким поясом растянувшись более чем на 2000 км между Каспийским морем на западе и горами Алтая на востоке». Удалите из строки все слова, начинающиеся на букву «м». Результат выведите на экран в виде строки.

2) Дан список случайных целых чисел. Замените все нечетные числа списка нулями, выведите их количество.

3) Список содержит положительные и отрицательные числа температуры в январе 2019 года в Алматы. Требуется положительные числа

поместить в один список, а отрицательные – в другой. Списки выведите на печать.

4) Напишите программу, которая запрашивает у пользователя две строки и формирует из них список. Если строки состоят только из чисел, то программа добавляет в середину списка сумму введенных чисел. Иначе добавляется строка, образованная из слияния двух введенных ранее строк. Итоговая строка выводится на экран.

5) Найдите произведение элементов списка, кратных 6 и оканчивающихся на 8. Если таких элементов нет, то сообщите об этом.

6) Напишите программу-игру. Компьютер загадывает случайное число, пользователь пытается его угадать. Пользователь вводит числа до тех пор, пока не угадает или не введет слово «Выход». Компьютер сравнивает число с введенным и сообщает пользователю, больше оно или меньше загаданного.

7) Дан произвольный текст. Определите в нем номер первого самого длинного слова.

8) Дан произвольный текст. Напечатайте все имеющиеся в нем цифры, определите их количество, сумму и найдите максимальное.

4.3. Перестановка элементов

Вы научитесь:

– создавать программы на языке программирования Python (пайтон) с использованием одномерных массивов.

Ключевые слова

Перестановка

Факториал

Дополнительная переменная

Орын ауыстыру

Факториал

Қосымша айнымалы

Retirement

Factorial

Additional variable



Дайте общую трактовку ключевых слов и предположите, как они будут фигурировать в последующем тексте по теме.



Большинство задач на обработку списка сводятся к комбинированию следующих алгоритмов (рис. 4.8).

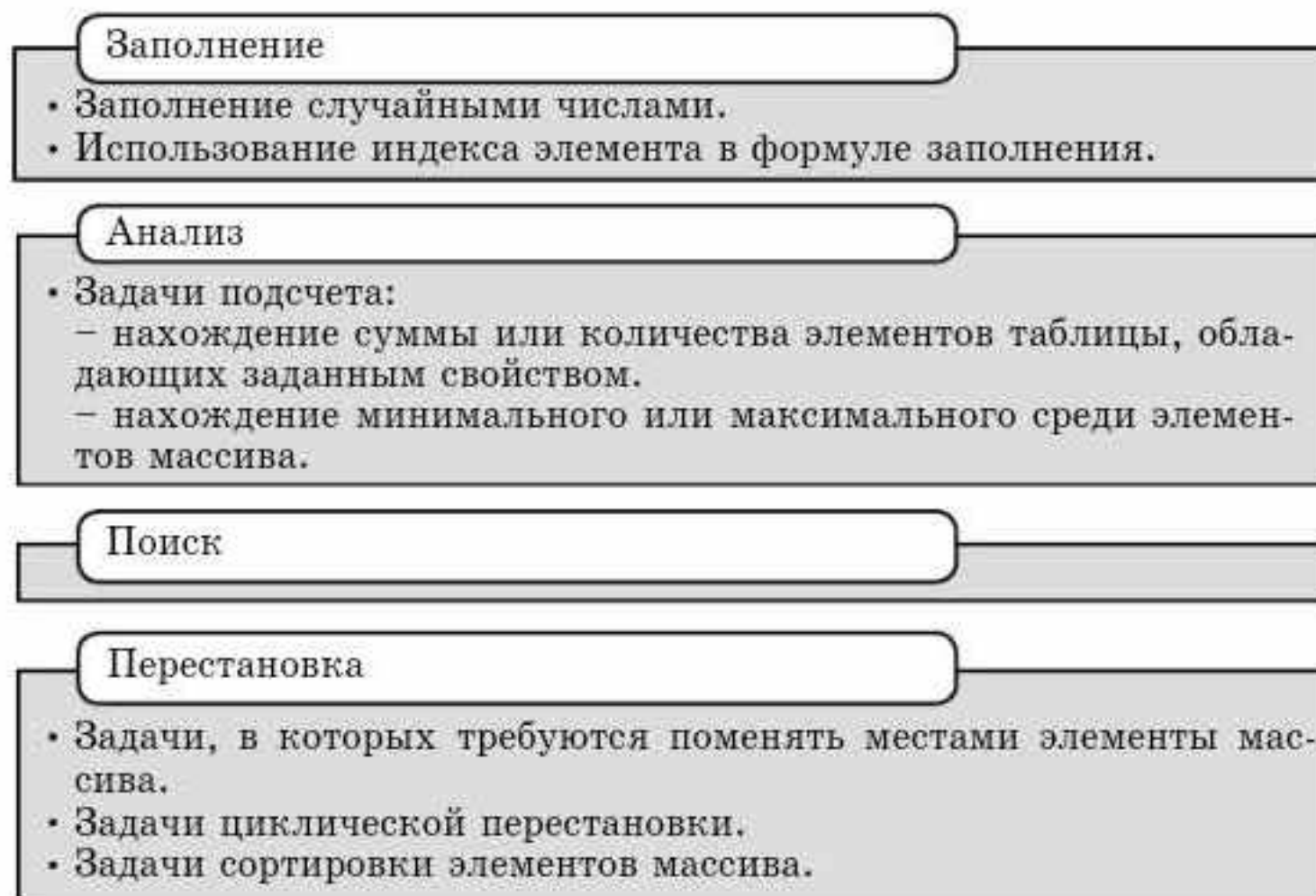


Рис. 4.8. Алгоритмы для обработки списков



Перестановками называются выборки элементов, отличающиеся только порядком расположения элементов, но не самими элементами.

Число перестановок на множестве из n элементов определяется по формуле: $P_n = n \cdot (n-1) \cdot (n-2) \dots 3 \cdot 2 \cdot 1 = n!$

$n!$ (n -факториал) – обозначение для краткой записи произведения всех натуральных чисел от 1 до n включительно. Например, перестановка пяти книг будет равна: $P_5 = 5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$.



Приведите примеры из жизни, когда в списках используются перестановки элементов.



Алгоритм перестановки очень часто применяется при решении практических задач. Каким образом происходит перестановка элементов в списке (массиве)?

Наглядный способ для понимания – представить переменную в виде кружки с чаем. Чтобы поменять содержимое кружки (рис. 4.9), нужно использовать третью пустую кружку в качестве посредника или дополнительной переменной.



Рис. 4.9. Перестановка местами

Обозначим переменными **a[1]** и **a[2]** две кружки с чаем. Поменяем содержимое кружек между собой посредством третьей кружки, обозначенной как переменная **k**. Перельем в третью кружку **k** содержимое второй кружки **a[2]**, тогда получим **k = a[2]**. В освобожденную кружку **a[2]** перельем содержимое кружки **a[1]**, т. е. получим **a[2] = a[1]**. Переменная **a[1]** освободится, в нее переливаем содержимое кружки-посредника **k**, получим: **a[1] = k**. Таким образом, содержимое кружек поменяется, соответственно, элементы меняются местами.

Рассмотрим перестановку генерирования комбинаций элементов последовательности abc. В результате должны получиться следующие списки последовательностей: abc, acb, bac, bca, cab, cba. Используем простой способ перестановки элементов в списке с применением инструмента itertools.permutations из стандартной библиотеки.	1-й способ <pre>import itertools print(list(itertools.permutations(['a', 'b', 'c'])))</pre> Результат <pre>[('a', 'b', 'c'), ('a', 'c', 'b'), ('b', 'a', 'c'), ('b', 'c', 'a'), ('c', 'a', 'b'), ('c', 'b', 'a')]</pre> 2-й способ <pre>import itertools for i in itertools.permutations('abc',3): print(''.join(i))</pre> Результат <pre>abc bca acb cab bac cba</pre>
---	--

В предыдущей теме вы решали задачи, в которых нужно было найти максимальное и минимальное значения списка. А как можно поменять их местами?



Попробуйте решить задачу, состоящую из двух частей:
 1) поиск минимума и максимума (их индексов) в списке;
 2) перестановка местами минимума и максимума в списке.



Шаг за шагом

Выполните первую часть – поиск минимума и максимума в списке.

1. Присвойте переменной первый индекс списка.
2. Переберите все элементы списка в цикле, начиная со второго. Проверьте условие: если второй элемент списка меньше первого, то переменной нужно присвоить текущий индекс.
3. Поиск максимума выполняется аналогично. Проверяется, если второй элемент списка больше первого, то переменной нужно присвоить текущий индекс.

Выполните вторую часть – перестановка местами минимума и максимума списка.

1. Присвойте буферной (дополнительной) переменной значение минимума.
2. Запишите по индексу минимума максимум списка.
3. Запишите по индексу максимума значение, хранимое в буферной переменной.

Код программы

```
# создание списка из 20 случайных чисел в диапазоне от 0 до 100
from random import random
n = 20
array = [0]*n
for i in range(n):
    array[i] = int(random()*100)
    print(array[i],end=' ')
print()
mini = 0
maxi = 0
# поиск минимального и максимального элементов списка
for i in range(n):
    if array[i] < array[mini]:
        mini = i
    elif array[i] > array[maxi]:
        maxi = i
```

```

print('array[%d]=%d array[%d]=%d' % (mini+1, array[mini], maxi+1,
array[maxi]))
# перестановка местами минимального и максимального элемен-
тов списка
buffer = array[mini]
array[mini] = array[maxi]
array[maxi] = buffer
for i in range(20):
    print(array[i],end=' ')
print()

```

Ниже представлены откомпилированная программа и результат.

```

File Edit Format Run Options Window Help
# создание списка 20 случайных чисел в диапазоне от 0 до 100
from random import random
n = 20
array = [0]*n
for i in range(n):
    array[i] = int(random()*100)
    print(array[i],end=' ')
print()
mini = 0
maxi = 0
# поиск минимального и максимального элементов списка
for i in range(n):
    if array[i] < array[mini]:
        mini = i
    elif array[i] > array[maxi]:
        maxi = i
print('array[%d]=%d array[%d]=%d' % (mini+1, array[mini], maxi+1, array[maxi]))
# перестановка местами минимального и максимального элементов списка
buffer = array[mini]
array[mini] = array[maxi]
array[maxi] = buffer
for i in range(20):
    print(array[i],end=' ')
print()

=== RESTART: C:/Users/Алира/Desktop/Задачи Питон/перестановка max и min.py ===
67 38 4 75 57 21 69 37 31 94 25 58 23 24 77 74 19 62 20 95
array[3]=4 array[20]=95
67 38 95 75 57 21 69 37 31 94 25 58 23 24 77 74 19 62 20 4
>>>

```

Проанализируйте.

Можно ли применить представленную ниже программу для перестановки минимального и максимального элементов в списке? Ответ прокомментируйте. Дайте оценку двум способам решения одной задачи.



```

from random import random
n = 20
array = [0]*n
for i in range(n):
    array[i] = int(random()*100)
    print(array[i],end=' ')
print()
mini = min(array)
maxi = max(array)
imini = array.index(mini)
imaxi = array.index(maxi)
print('array[%d]=%d array[%d]=%d'%(imini+1,mini,imaxi+1,maxi))
array[imini],array[imaxi] = array[imaxi],array[imini]
for i in range(20):
    print(array[i],end=' ')
print()

```



Знание и понимание

1. Как вы понимаете термин «перестановка»?
2. Когда применяются перестановки в списках (массивах)?
3. По какой формуле можно определить количество перестановок? Сколько может быть возможных перестановок n предметов из комбинаторики (область математики)?
4. Почему для перестановки элементов списка используется третья переменная? Приведите пример из жизни.
5. Дан целочисленный список, состоящий из трех элементов – 1, 2, 3. Сколько возможных перестановок можно выполнить для трех элементов? Для четырех?
6. В списке, состоящем из 10 элементов $a=[1, 3, 5, 7, 9, 8, 6, 4, 2, 10]$, нужно поменять местами пятый и девятый элементы. Правильно ли записан фрагмент перестановки элементов списка? Запишите новый список с переставленными элементами.
 $b = a[5]$
 $a[5] = a[9]$
 $a[9] = b$
7. Сколько и какие сочетания перестановок будут выведены на экран после выполнения программы?

```
import itertools
a=[0,1,2]
print(list(itertools.permutations(a)))
```
8. Составьте и запишите свою версию рассказа, употребив все предложенные ключевые термины по теме.



и максимальный расходы электроэнергии, используя обработку списка. Также узнайте, на сколько процентов отличаются минимальный и максимальный расходы от среднемесячного, и поменяйте их местами.



Задания по тематике окружающего мира.

1) Десять экологов сдали экзамен по охране окружающей среды Восточно-Казахстанской области. Найдите в массиве, сколько экологов будут охранять среду отлично, сколько – хорошо, а сколько – удовлетворительно.

2) Школьники на уроке физической культуры встают в круг и начинают считалку. Каждый раз, после того как выбывает n -й ребенок, круг смыкается. Считалка повторяется, пока не останется один ребенок. Обработайте список, чтобы узнать, кто останется.

3) Показатели дневной и ночной температуры воздуха проверяются ежедневно и записываются в таблицу. Если в течение трех дней подряд показатели среднесуточной температуры составляют менее 8 градусов Цельсия, начинается отопительный сезон. Определите, какого числа в этом году был начат отопительный сезон, если известно, что это произошло в текущем месяце. Используйте обработку массивов.

4) Роща ценных деревьев согдийского ясеня расположена в живописном уголке Шарынского каньона. Роща выжила после ледникового периода 10 тыс. лет назад. Известны координаты каждого дерева. Требуется огородить рощу от браконьеров забором прямоугольной формы, потратив на это как можно меньше денег. Найдите периметр охватывающего прямоугольника, обработав массив (список).

4.4. Двумерный массив

Вы научитесь:

– создавать программы на языке программирования Python с использованием двумерных массивов.

Ключевые слова

Двумерный массив

Матрица

Строка

Столбец

Екі өлшемді жиым

Матрица

Жол

Баған

Two-dimensional array

Matrix

Line

Column



Дайте общую трактовку ключевых слов и предположите, как они будут фигурировать в последующем тексте по теме.



В § 4.1 вы познакомились с одномерными массивами. В зависимости от количества измерений массивы бывают **одномерные**, **двумерные**, **n-мерные**. В языке программирования Python таблицу можно представить в виде **списка строк**.

Это могут быть таблицы результатов экзамена, умножения, расписание самолетов, поездов и т. д.



Данные, хранящиеся в прямоугольной таблице, называются **матрицами**, или **двумерными массивами** (two-dimensional array).

Двумерный массив является массивом ссылок на одномерные массивы. Как вам известно, элементы одномерного массива индексируются одиночным целым числом. А элементы двумерного массива индексируются двумя целыми числами. Первое число – это определение строки, второе – столбца.

После создания массива любой его объект можно использовать в программе везде, где используется имя переменной. Для этого указывают имя массива и целочисленный индексом в квадратных скобках.

Для примера создадим числовую таблицу, состоящую из двух строк и трех столбцов. В одной строке записано `arr = [[11, 22, 33], [44, 55, 66]]`.

Первая строка списка <code>arr[0] = [11, 22, 33]</code>	Вторая строка списка <code>arr[1] = [44, 55, 66]</code>
<code>arr[0][0] = 11</code>	<code>arr[1][0] = 44</code>
<code>arr[0][1] = 22</code>	<code>arr[1][1] = 55</code>
<code>arr[0][2] = 33</code>	<code>arr[1][2] = 66</code>

В двумерных массивах (списках) для **обработки** и **вывода** данных используются два вложенных цикла:

- по **номеру строки** – первый цикл;
- по **элементам внутри строки** – второй цикл.

Самый простой вывод матрицы можно осуществить одним оператором `print(a)`. Рассмотрим примеры для вывода двумерного массива на экран.



Необходимо **построчно** вывести на экран двумерный массив (список). Массив состоит из чисел, разделенных внутри одной строки пробелами.

Проанализируйте, каким образом осуществляется работа первого и второго циклов в кодах программ 1, 2. Чем они различаются?

Шаг за шагом

Код программы 1. Цикл по индексу списка	Код программы 2. Цикл по значениям списка
<pre>arr=[[11, 22, 33], [44, 55, 66]] for i in range(len (arr)): for j in range(len (arr[i])): print("arr[",i,j,"]=",arr[i][j], end=' ') print() ===== RESTART: C:/Users arr[0 0]= 11 arr[0 1]= 22 arr[0 2]= 33 arr[1 0]= 44 arr[1 1]= 55 arr[1 2]= 66 >>></pre>	<pre>arr=[[11, 22, 33], [44, 55, 66]] for row in arr: for elem in row: print(elem, end=' ') print() ===== RESTART: C:/Users/Алира 11 22 33 44 55 66 >>></pre>



В представленных кодах программ (1, 2) использованы два вложенных цикла для подсчета суммы всех чисел в матрице. Результаты выполнения двух программ одинаковые.

Проанализируйте программные коды и сделайте вывод, какая из них обрабатывает списки по индексу. Какая программа обрабатывает списки по значениям строк?



Код программы 1

```
arr=[[1,2,3], [4,5,6]]
sum = 0
for i in range (len(arr)):
    for j in range (len(arr[i])):
        sum += arr[i][j]
        print (sum, end=' ')

===== RESTART: C:/Users
1 3 6 10 15 21
>>>
```

Код программы 2

```
arr=[[1, 2, 3], [4, 5, 6]]
sum = 0
for row in arr:
```

```
for elem in row:
    sum += elem
    print(sum, end=' ')
```

```
===== RESTART: C:/Users
1 3 6 10 15 21
>>> |
```



Знание и понимание



1. Что вы понимаете под термином «двумерный массив»?
2. Сколько измерений имеет двумерный массив?
3. В виде чего можно представить таблицу в языке программирования Python?
4. Каким образом индексируется двумерный массив?
5. Приведите примеры двумерных массивов из жизни.
6. Сколько циклов нужно использовать в двумерном массиве при обработке и выводе данных?
7. Пусть дан двумерный массив, состоящий из n строк таблицы. Каждая строка в свою очередь содержит m чисел, разделенных пробелами. Каким образом можно вводить и выводить данные?
8. Составьте и запишите свою версию рассказа, употребив все предложенные ключевые термины по теме.



Применение

Анализ



9. Дан квадратный массив из n строк и n столбцов. Необходимо элементам, находящимся на главной диагонали, присвоить значение 1. Главная диагональ проходит из левого верхнего угла в правый нижний угол. Элементам, которые находятся выше главной диагонали, присвоить значение 0. А элементам, находящимся ниже главной диагонали, – значение 2. Приведем квадратный массив для $n = 4$:

Индексы массива				Элементы массива			
[0],[0]	[0],[1]	[0],[2]	[0],[3]	1	0	0	0
[1],[0]	[1],[1]	[1],[2]	[1],[3]	2	1	0	0
[2],[0]	[2],[1]	[2],[2]	[2],[3]	2	2	1	0
[3],[0]	[3],[1]	[3],[2]	[3],[3]	2	2	2	1

Например, $A[0], [0]=1$; $A[1], [1]=1$; $A[2], [2]=1$; $A[3], [3]=1$.

По условию задачи элементы, которые лежат выше главной диагонали, – это элементы $A[i][j]$, для которых $i < j$. Для элементов ниже

главной диагонали – $i > j$. Таким образом, сравнивая значения i и j , определяем значение $A[i][j]$.

Ниже представлены два алгоритма решения задачи. Какой из способов будет лучшим?

1-й способ

```
for i in range(n):
    for j in range(n):
        if i < j:
            A[i][j] = 0
        elif i > j:
            A[i][j] = 2
        else:
            A[i][j] = 1
```

2-й способ

```
for i in range(n):
    A[i][i] = 1
for i in range(n):
    for j in range(i + 1, n):
        A[i][j] = 0
for i in range(n):
    for j in range(0, i):
        A[i][j] = 2
```

10. Дано число n . Создайте массив размером $n \times n$ и заполните его по следующему правилу:

- числа на диагонали, идущей из правого верхнего в левый нижний угол, равны 1.
- числа, стоящие выше этой диагонали, равны 0.
- числа, стоящие ниже этой диагонали, равны 2.

Полученный массив выведите на экран. Числа в строке разделяйте одним пробелом.

Примеры

Входные данные

4

Выходные данные

```
0 0 0 1
0 0 1 2
0 1 2 2
1 2 2 2
```

11. Пусть задан массив, в котором $n = 5$ строк, $m = 6$ столбцов. Элементы в массиве разделены пробелами:

```
0 0 0 0 0 0
0 1 2 3 4 5
0 2 4 6 8 10
0 3 6 9 12 15
0 4 8 12 16 20
```

Создайте программу, вычисляющую элементы в строке i и столбце j по формуле: $A[i][j]=i*j$. Для создания массива используйте генератор: `[[i*j for j in range(m)] for i in range(n)]`.



Синтез

Оценивание

12. С трех метеостанций, расположенных в разных регионах Казахстана, получены значения температуры воздуха за 4 дня.

Номер станции	1-й день	2-й день	3-й день	4-й день
1	-10	-12	-18	-15
2	15	18	20	17
3	5	7	11	13

В двумерном массиве запись показаний выглядит следующим образом:

$t[1],[1]=-10$	$t[1],[2]=-12$	$t[1],[3]=-18$	$t[1],[4]=-15$
$t[2],[1]=15$	$t[2],[2]=18$	$t[2],[3]=20$	$t[2],[4]=17$
$t[3],[1]=5$	$t[3],[2]=7$	$t[3],[3]=11$	$t[3],[4]=13$

- 1) Напишите программу, которая должна распечатать показатели температуры на 2-й метеостанции за 4-й день и на 3-й метеостанции за 1-й день.
- 2) Распечатайте показания термометров всех метеостанций за 2-й день.
- 3) Определите среднюю температуру на 3-й метеостанции.
- 4) Распечатайте, в какие дни и на каких метеостанциях температура была в диапазоне 15–20 градусов тепла.

13. В кинотеатре n рядов по m мест в каждом. В двумерном массиве хранится информация о проданных билетах. Число 1 означает, что билет на данное место уже продан, число 0 – что место свободно. Поступил запрос на продажу k билетов на соседние места в одном ряду. Определите, можно ли выполнить такой запрос.

Входные данные

Данные вводятся с клавиатуры и выводятся на экран. Программа получает на вход числа n и m . Далее идет n строк, которые содержат m чисел (0 или 1), разделенных пробелами. Затем дано число k .

Выходные данные

Программа должна вывести номер ряда, в котором есть k подряд идущих свободных мест. Если таких рядов несколько, то выведите номер наименьшего подходящего ряда. Если подходящего ряда нет, выведите число 0.

Примеры

Входные данные

```
3 4
0 1 0 1
1 0 0 1
1 1 1 1
2
```

Выходные данные

```
2
```

14. Дано нечетное число n . Создайте двумерный массив из $n \times n$ элементов, заполнив его символами "." (каждый элемент массива является строкой из одного символа). Затем заполните символами "*" среднюю строку массива, средний столбец массива, главную и побочную диагонали. В результате единицы в массиве должны образовывать изображение звездочки. Выведите полученный массив на экран, разделяя элементы массива пробелами.

Примеры

Входные данные

```
5
```

Выходные данные

```
* . * . *
. * * * .
* * * * *
. * * * .
* . * . *
```

15. Даны два числа n и m . Создайте массив $n \times m$ и заполните его по следующим правилам:

– числа, стоящие в строке 0 или в столбце 0, равны 1 ($A[0][j]=1$, $A[i][0]=1$);

– для всех остальных элементов массива $A[i][j]=A[i-1][j]+A[i][j-1]$, т. е. каждый элемент равен сумме двух элементов, стоящих слева и сверху от него.

Входные данные

Вводятся два числа n и m .

Выходные данные

Выведите данный массив на экран, отводя на вывод каждого элемента массива ровно 6 символов (см. пример).

Примеры

Входные данные

4 6

Выходные данные

1	1	1	1	1	1
1	2	3	4	5	6
1	3	6	10	15	21
1	4	10	20	35	56



Некоторые правила на языке программирования Python, которых следует обязательно придерживаться.

- 1) Пробелам придают особое значение для удобного чтения кода программы.
- 2) Пробелы имеют синтаксическое значение.
- 3) Используются пробелы для создания отступов, а не символы табуляции.
- 4) Для каждого уровня синтаксически значимых отступов, используйте по четыре пробела.
- 5) Длина строк не должна превышать 79 символов.
- 6) Дополнительные строки, которые являются продолжением длинных выражений, должны выделяться четырьмя дополнительными пробелами.
- 7) Индексы элементов списков не должны быть окружены пробелами.
- 8) До и после оператора присваивания помещайте только по одному пробелу.

4.5. Сортировка

Вы научитесь:

- применять алгоритмы сортировки;
- создавать программы на языке программирования Python с использованием одномерных массивов;
- создавать программы на языке программирования Python с использованием двумерных массивов.

Ключевые слова

Сортировка пузырьком	Көпіршік әдісі бойынша сұрыптау	Bubble sort
Сортировка по возрастанию	Өсуі бойынша сұрыптау	Sort Ascending
Сортировка по убыванию	Кемуі бойынша сұрыптау	Sorting in decreasing order



Дайте общую трактовку ключевых слов и предположите, как они будут фигурировать в последующем тексте по теме.



Сортировка элементов в массиве является неотъемлемой частью анализа данных.



Сортировка – упорядочение элементов в подмножестве данных по какому-либо критерию.



Основная цель сортировки – облегчить поиск элементов в отсортированном массиве при обработке данных. Сортировка бывает двух видов:

1) **сортировка по убыванию**, при котором следующий элемент не больше предыдущего: 7, 6, 5, 4, 3, 2, 1;

2) **сортировка по возрастанию**, при которой каждый последующий элемент не меньше предыдущего: 1, 2, 3, 4, 5, 6, 7.

Существует большое количество различных алгоритмов сортировки, отличающиеся друг от друга временем выполнения, сложностью в понимании и реализации, используемой памятью (рис. 4.10).

Сортировка

- Метод «пузырька»
- Метод простого выбора
- Метод простого включения
- Метод Шелла
- Пирамидальная
- Быстрая сортировка Хоара

Рис. 4.10. Виды сортировок

Разберем один из самых простых алгоритмов сортировки – сортировка методом «пузырька». Этот метод очень прост в понимании и реализации. Подходит для сортировки небольших списков (массивов).

Алгоритм сортировки заключается в следующем:

1. Для сортировки по возрастанию сравниваем $[i]$ элемент списка с $[i+1]$. Если больший элемент имеет наименьший порядковый номер, то они меняются местами.

2. При упорядочивании самый большой элемент сдвигается в конец списка.

3. Процесс повторяется до последнего отсортированного элемента списка.



Дан список $[1, 6, 9, 7, 8, 3]$. Расположите элементы списка в порядке убывания. То есть, если элемент слева меньше элемента-соседа справа, то меняется с ним местами. В итоге должен получиться список $[9, 8, 7, 6, 3, 1]$.



Шаг за шагом

В табл. 4.6 приведен последовательный разбор сортировки методом «пузырька».

Таблица 4.6

Первая итерация							Выполним сортировку по убыванию списка $[1, 6, 9, 7, 8, 3]$ вручную. После прохождения первой итерации (прохода) минимальное число 1 переходит на конец списка. Поэтому после первой итерации с числом 1 сравнивать нет смысла.
Сравниваем 1 и 2	6	1	9	7	8	3	
Сравниваем 2 и 3	6	9	1	7	8	3	
Сравниваем 3 и 4	6	9	7	1	8	3	
Сравниваем 4 и 5	6	9	7	8	1	3	
Сравниваем 5 и 6	6	9	7	8	3	1	
Вторая итерация							После прохождения второй итерации в конце списка будут находиться два числа из списка по убыванию: 3 и 1. Что вы заметили? Количество сравнений уменьшилось на единицу.
Сравниваем 1 и 2	9	6	7	8	3	1	
Сравниваем 2 и 3	9	7	6	8	3	1	
Сравниваем 3 и 4	9	7	8	6	3	1	
Сравниваем 4 и 5	9	7	8	6	3	1	
Третья итерация							Продолжаем сравнивать элементы в списке. После третьей итерации в конце списка будут находиться числа 6, 3, 1.
Сравниваем 1 и 2	9	7	8	6	3	1	
Сравниваем 2 и 3	9	8	7	6	3	1	
Сравниваем 3 и 4	9	8	7	6	3	1	

Четвертая итерация						
Сравниваем 1 и 2	9	8	7	6	3	1
Сравниваем 2 и 3	9	8	7	6	3	1
Пятая итерация						
Сравниваем 1 и 2	9	8	7	6	3	1
[1, 6, 9, 7, 8, 3] → [9, 8, 7, 6, 3, 1]						

После прохождения пятой итерации получим отсортированный список по убыванию.

Если список состоит из шести элементов, то выполняется пять итераций для последовательного сравнения соседних элементов.

Программа сортировки методом «пузырька»

```
# Список из шести чисел
lst = [6, 9, 7, 8, 3, 1]
# Определение длины списка
n = len(lst)
# Внешний цикл для отсчета количества итераций по списку
for j in range(0, n-1):
    # Вложенный цикл для сравнения i-го с i+1-м элементом
    # Перестановка элементов местами при необходимости
    # Количество сравнений уменьшается на величину j с каждой итерацией
    for i in range(0, n-j-1):
        if lst[i] < lst[i+1]:
            lst[i], lst[i + 1] = lst[i + 1], lst[i]
    print(j+1, "-ая итерация цикла - ", end=" ")
    print(lst)
print(lst)
```

Результат

```
=== RESTART: C:/Users/Алира/Desktop/Задачи Питон
1 -ая итерация цикла - [6, 9, 7, 8, 3, 1]
2 -ая итерация цикла - [9, 7, 8, 6, 3, 1]
3 -ая итерация цикла - [9, 8, 7, 6, 3, 1]
4 -ая итерация цикла - [9, 8, 7, 6, 3, 1]
5 -ая итерация цикла - [9, 8, 7, 6, 3, 1]
[9, 8, 7, 6, 3, 1]
>>>
```

В языке программирования Python легко выполнять сортировку, применяя один из двух способов вызова сортировки.

Первый способ – это метод `sort([key])`, который сортирует элементы по возрастанию.

Код программы

```
arr = [13, -2, 55, 17, -15, 0, 8, 3, 10, 11]
arr.sort()
print (arr)
```

Результат

```
===== RESTART: C:/Users/Алира/Desktop/2
[-15, -2, 0, 3, 8, 10, 11, 13, 17, 55]
>>>
```

Приведем еще пример, в котором элементы списка являются вложенными списками (матрица). В такой матрице сортировка происходит по первым элементам или по первому столбцу.

```
arr = [[35, 100], [18, 8], [31, 11]]
arr.sort()
print (arr)
```

Результат

```
[[18, 8], [31, 11], [35, 100]]
```

Метод `sort` не возвращает значения, поэтому результат данного метода нельзя использовать в арифметических выражениях или при выводе результата.

Второй способ – это функция `sorted`, наоборот, не модифицирует переданный ей список, а возвращает новый список. Данную функцию можно использовать следующим образом:

```
import random
a = [random.randint(1, 10) for i in range(10)]
b = sorted(a)
print(b)
```

```
===== RESTART: C:/Users/Ar
[3, 3, 5, 6, 6, 7, 8, 8, 8, 10]
```



В чем преимущество сортировки списков с помощью функции `sort()`? Исследуйте функцию `sorted`. Существует ли возможность провести в одной строке сортировку списка, состоящего из чисел, и вывести его результат на экран?



Дана матрица 3×3 , элементы в ней созданы случайным образом. Необходимо матрицу отсортировать по возрастанию элементов первой строки. В приведенной ниже программе, запишите комментарии к каждому оператору. Что получится в результате работы программы? Как отсортирована первая строка? Как перемещены элементы столбцов, упорядоченные по возрастанию?



Шаг за шагом

В матрице меняется последовательность столбцов, т. е. элементы первой строки будут отсортированы по возрастанию.

Код программы

```
from random import random
n1 = 3
n2 = 3
a = []
for i in range(n1):
    b = []
    for j in range(n2):
        n = int(random() * 100)
        b.append(n)
        print("%4d" % n, end="")
    print()
    a.append(b)
print()
z = n2-1
while z > 0:
    index = 0
    for j in range(z+1):
        if a[0][j] > a[0][index]:
            index = j
    for i in range(n1):
        buf = a[i][index]
        a[i][index] = a[i][z]
        a[i][z] = buf
    z -= 1
for i in range(n1):
    for j in range(n2):
        print("%4d" % a[i][j], end="")
    print()
```

Результат

```
===== RESTART:
59  64  55
20  83  72
 4   5  64

55  59  64
72  20  83
64   4   5
```

Для того чтобы быстро расставить все элементы в списке в обратном порядке, применяют функцию **reverse()**. Приведем пример.

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9]
data.reverse()
print(data)
```

```
===== RESTART: C:/Users/
[9, 8, 7, 6, 5, 4, 3, 2, 1]
>>>
```



Знание и понимание

1. Что такое *сортировка*?
2. Какая сортировка называется сортировкой по убыванию? Возрастающую?
3. Какова основная цель сортировки?
4. Какие виды сортировок существуют?
5. Какой метод сортировки самый распространенный?
6. Как происходит перебор элементов массива методом «пузырька»?
7. С какой целью применяются функции для сортировки списков `sort()` и `sorted`?
8. Дан код программы. Что будет выведено на экран в результате ее выполнения?

```
kol_a = 2
kol_b = 3
elem_arr = 0
arr = []
for i in range(kol_a):
    arr.append([])
    for j in range(kol_b):
        arr[i].append(elem_arr)
        elem_arr += 1
print(arr)
```

9. Дан код программы:

```
list_arr = ['список', 'одномерный массив', 'двумерный массив',
            'строка', 'столбец', 'цикл']
for i in list_arr:
    print(i)
```

Какой результат будет выведен на экран? Какой размерности список?

10. Составьте и запишите свою версию рассказа, употребив все предложенные ключевые термины по теме.



Применение

Анализ

11. В приведенной программе дана матрица 5×6 . Выполните по шагам каждый оператор, записав комментарии к ним. Каково условие задачи, что должно получиться в результате выполнения программы? Сколько раз в программе применяется генератор? Выполните программу в среде программирования Python.

```
from random import random
N = 5
M = 6
lst = []
```

```

arr = [0] * N*M
for i in range(N):
    arr1 = []
    for j in range(M):
        arr1.append(int(random()*100))
    lst.append(arr1)
for i in range(N):
    for j in range(M):
        print("% 3d" % lst[i][j], end='')
    print()
print()
k = -1
for j in range(M):
    for i in range(N):
        k += 1
        arr[k] = lst[i][j]
        print("% 3d" % arr[k], end='')
print()

```

12. В одномерном массиве, созданном случайным образом из n целых чисел, отсортируйте элементы массива методом выбора по возрастанию. Сортировка выбором заключается в поиске максимального элемента массива в каждой итерации. Максимальный элемент перемещается на последнее место в просматриваемой части массива.

Пример, входного и выходного массива, созданных случайным образом:

Входной массив

[47, 12, 51, 54, 16, 24, 74, 86, 99, 30]

Выходной массив

[12, 16, 24, 30, 47, 51, 54, 74, 86, 99]

13. Создайте программу, состоящую из списка целых чисел. Проведите сортировку по возрастанию суммы цифр чисел.

14. Дан список целых чисел, состоящий из 30 элементов. Найдите сумму пяти самых больших и пяти самых маленьких элементов списка.



Синтез

Оценивание



15. Дан двумерный массив из целых чисел, состоящий из пяти строк и четырех столбцов. Найдите сумму пяти самых больших и пяти самых маленьких элементов списка.

16. Создайте список, сформированный из целых чисел. Замените все нечетные числа списка нулями и выведите их количество на печать.



Интересные задачи

1. «Любимые вещи». Создайте список своих любимых развлечений и сохраните его в переменной `games`. Затем создайте список любимых лакомств, сохранив его в переменной `foods`. Объедините два этих списка, сохранив результат в переменной `favorites`, и напечатайте значение этой переменной.

2. «Подсчет воинов». Есть 3 дома, на крыше каждого из которых прячутся по 25 ниндзя, и есть 2 туннеля, в каждом из которых скрывается по 40 самураев. Сколько всего воинов?

3. «Вариация массива». Кайрат назвал «вариацией массива» операции, выполненные по следующему алгоритму:

- 1) массив сортируется по возрастанию элементов;
- 2) после сортировки находится разность второго и первого элементов, третьего и второго, четвертого и третьего и т. д., до разности последнего и предпоследнего элементов;
- 3) затем все найденные разности суммируются.

Помогите Кайрату, написать программу, которая проделывает указанную операцию над заданным массивом.

Формат входа. В первой строке задано целое число n – количество элементов в массиве ($2 \leq n \leq 1000$). В следующей строке через пробел заданы целые числа, каждое из которых по модулю не превосходит 10^6 .

Формат выхода. Выведите единственное целое число – вариацию заданного набора чисел.

Пример

Вход	Выход
4	5
5 2 6 1	

Примечание. Результат получается следующий: отсортированный массив: 1 2 5 6; вычисляем разности: $2 - 1 = 1$, $5 - 2 = 3$, $6 - 5 = 1$; вычисляем сумму разностей $1 + 3 + 1 = 5$.

4.6. Удаление и вставка элемента

Вы научитесь:

- создавать программы на языке программирования Python с использованием одномерных массивов;
- создавать программы на языке программирования Python с использованием двумерных массивов.

Ключевые слова

Удаление элемента

Элементті жою

Delete an item

Вставка элемента

Элементті кірістіру

Inserting an element



Дайте общую трактовку ключевых слов и предположите, как они будут фигурировать в последующем тексте по теме.



Часто из массива необходимо удалять ненужные элементы. Каким образом происходит удаление элемента из массива?



Удаление элемента из массива означает, что на место удаленного элемента записывается тот элемент массива, который должен остаться. Поэтому элементы массива сдвигаются к какому-нибудь краю массива. Обычно сдвиг делается в сторону начала массива, чтобы индексация снова начиналась с нуля. Рассмотрим удаление на конкретном примере.



Как удалить элементы списка по условию? Например, из списка баллов по экзаменам необходимо удалить элементы, значения которых больше 40 и меньше 60 баллов. При этом удаляемые числа необходимо сохранить в другом списке.



Шаг за шагом

Для удаления элемента списка воспользуемся инструкцией **del**, указав сам список и индекс удаляемого элемента. Алгоритм решения задачи заключается в переборке элементов списка.

- Если элемент списка удовлетворяет условию, то удаляем элемент.
- При удалении элемента на его место становится следующий элемент.

– Переходим к следующему элементу, при этом пропускаем проверку элемента, который занял место удаленного.

Цикл **for** при удалении из списка элементов нельзя использовать, поскольку меняется количество элементов списка.

```
import random
arr = []
for i in range(10):
    n = round(random.random() * 100)
    arr.append(n)
print("Исходный список", arr)
lst = []
i = 0
while i < len(arr):
    if 20 < arr[i] < 40:
        lst.append(arr[i])
        del arr[i]
    else:
        i += 1
print("Список после удаления по условию", arr)
print("Список удаленных", lst)
```

Результат

Исходный список [10, 24, 46, 76, 32, 79, 19, 90, 12, 94]

Список после удаления по условию [10, 46, 76, 79, 19, 90, 12, 94]

Список удаленных [24, 32]

В программе использован цикл **while**, который позволяет измерять длину списка на каждой итерации.



Не менее часто применяется на практике **вставка** элементов в массив. Как можно вставлять элементы в массив?



При добавлении элементов в массив их количество будет возрастать. Данные, удовлетворяющие заданному условию, можно добавлять как перед, так и после элемента массива. Это зависит от условия решаемой задачи.

Для вставки элементов в списки применяется метод **insert(index, x)**. Данный метод добавляет элемент **x** в список по индексу **index**.



В список **colors**, состоящий из трех элементов строкового типа данных, во вторую позицию нужно вставить новый элемент.



Шаг за шагом

Код программы

```
colors = ['blue', 'orange', 'red']
colors.insert(2, 'yellow') # вставляет элемент в указанную позицию
print(colors)
```

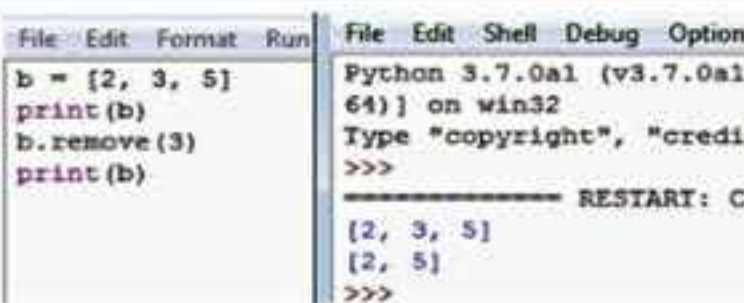
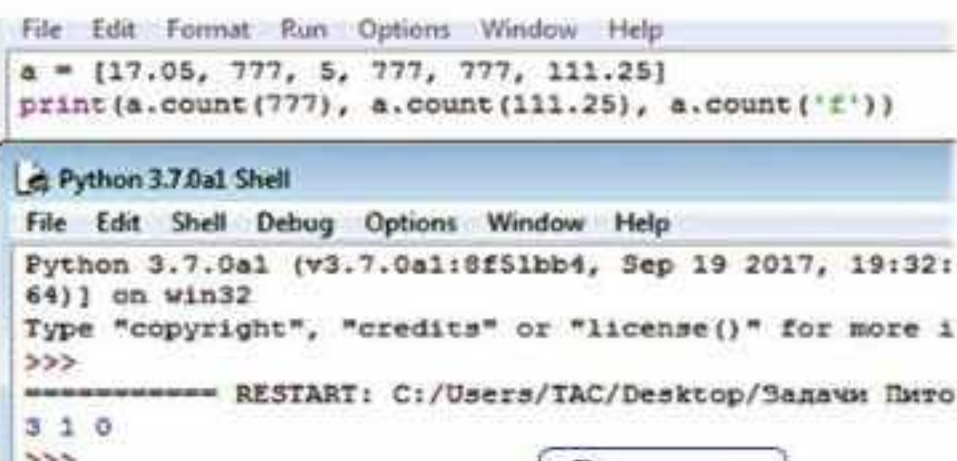
Результат

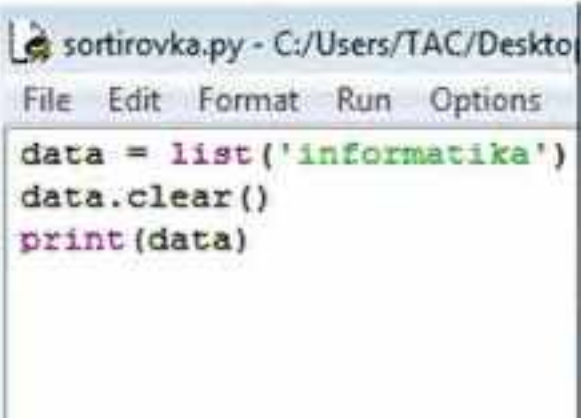
```
===== RESTART: C:/Users/TAC/Desktop
['blue', 'orange', 'yellow', 'red']
>>>
```

Кроме выше рассмотренных методов работы со списками, существует ряд других, которые приведены в табл. 4.7.

Таблица 4.7

Методы в списках

Методы	Назначение	Примеры
remove(x)	Удаляет только первое вхождение элемента x. Если элемент не найден, генерирует исключение ValueError.	<pre>b=[2, 3, 5] print(b) b.remove(3) print(b)</pre>  [2, 5] – будет удалено первое вхождение на данный элемент.
count(x)	Вычисляет количество элементов равных x	<pre>a = [17.05, 777, 5, 777, 777, 111.25] print(a.count(777), a.count(111.25), a.count('f')) # считает количество повторений</pre> Код программы  Результат

Методы	Назначение	Примеры	
clear()	Удаляет элементы списка	<pre>data = list('informatika') data.clear() print(data)</pre>	
		Код программы	Результат
			



Знание и понимание

1. Каким образом можно вставлять и удалять элементы массива?
2. Почему использование методов вставки и удаления элементов в списке облегчает работу программиста?
3. Разгадайте кроссворд по разделу IV «Программирование алгоритмов на языке программирования Python», записав слова по горизонтали и вертикали.

По горизонтали:

1. Чтобы поменять местами элементы массива, используется метод
4. ... данных с клавиатуры.
5. Наибольшее число в последовательности – это
9. Первая ..., которая выводит на экран строку текста, состоит из одной строки `print("Hello, world!")`.
10. ... – это повторение одного и того же участка кода в программе.
11. Какой метод используется, чтобы убрать ненужные элементы ... ?
12. Структура в Python, где элементы находятся в упорядоченной последовательности.
13. Как называется функция для печати на экран?
14. Упорядочение элементов в подмножестве данных по какому-либо критерию называется

По вертикали:

2. Отсортированный список [1, 2, 3, 5, 6, 7, 8] – это
3. Данные, хранящиеся в прямоугольной таблице, называются матрицами, или ... массивами.
6. ... представляется в языке программирования Python в виде списка строк.
7. Команда Run (F5) –